

综合练习

人员

郭栩睿、于家瑞、陶汇笙、王恩泽 到课

上周作业检查

上周作业链接: <https://cppoj.kids123code.com/contest/4301>

王向东老师周六十点半C++综合练习

刷新

#	用户名	姓名	编程分	时间	A	B	C	D
1	guoxurui	郭栩睿	400	432	100	100	100	100
2	yujiarui	于家瑞	400	491	100	100	100	100
3	yuxiaolong	于青龙	300	384	100	100	100	
4	hongchenqi	洪晨棋	300	454	100	100	100	
5	wangenze	王恩泽	200	73	100			100
6	taohuisheng	陶汇笙	200	77	100	100		
7	hongchendong	洪晨栋	200	102	100	100		

本周作业

<https://cppoj.kids123code.com/contest/4520> (课上讲了 A ~ D 这些题, 课后作业是 C 题, 学有余力做D、E)

课堂表现

部分同学们课上B题写的不好, 不太理解, 对于dij的理解不够深入, 课下建议再次复习理解本题, 于家瑞做得比较快。

课堂内容

杂务 (上周作业)

f[i]: 处理到点 i 时, 最少需要多少时间

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 1e4 + 5;
vector<int> vec[maxn];
int din[maxn], w[maxn], f[maxn];

int main()
{
    int n; cin >> n;
    for (int i = 1; i <= n; ++i) {
```

```

    int t; cin >> t;
    cin >> w[i];
    int x;
    while (true) {
        cin >> x;
        if (!x) break;
        vec[x].push_back(i); ++din[i];
    }
}

queue<int> q;
for (int i = 1; i <= n; ++i) {
    if (!din[i]) q.push(i), f[i] = w[i];
}
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int i : vec[u]) {
        --din[i]; f[i] = max(f[i], f[u]+w[i]);
        if (!din[i]) q.push(i);
    }
}

int res = 0;
for (int i = 1; i <= n; ++i) res = max(res, f[i]);
cout << res << endl;
return 0;
}

```

最短路计数

在 bfs 维护 dis 最短路的基础上, 额外维护一个 f 数组

其中, $f[i]$ 代表到 i 最短路的条数

如果 $u \rightarrow i$ 是第一次更新到 i 的最短路, 那么此时 $f[i] = f[u]$

否则如果 $u \rightarrow i$ 等于 i 的最短路, 那么此时 $f[i] += f[u]$

```

#include <bits/stdc++.h>
#define int long long

using namespace std;

const int maxn = 1e6 + 5;
const int mod = 100003;
const int inf = 0x3f3f3f3f3f3f3f3f;
int dis[maxn], f[maxn];
vector<int> vec[maxn];

signed main()
{

```

```

ios::sync_with_stdio(false);
cin.tie(0);

int n, m; cin >> n >> m;
while (m -- ) {
    int a, b; cin >> a >> b;
    vec[a].push_back(b), vec[b].push_back(a);
}

memset(dis, -1, sizeof(dis));
queue<int> q;
q.push(1); dis[1] = 0; f[1] = 1;
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int i : vec[u]) {
        if (dis[i] == -1) {
            q.push(i); dis[i] = dis[u]+1; f[i] = f[u];
        } else if (dis[i] == dis[u]+1) f[i] = (f[i] + f[u]) % mod;
    }
}

for (int i = 1; i <= n; ++i) cout << f[i] << "\n";
return 0;
}

```

路径统计

此题有重边, 需要先对边进行去重操作

解题方法跟上道题一模一样, 就是把用 bfs 求最短路的过程, 替换成用 dijkstra 求最短路

```

#include <bits/stdc++.h>

using namespace std;

const int maxn = 2000 + 5;
const int inf = 0x3f3f3f3f;
struct eInfo {
    int to, value;
};
vector<eInfo> vec[maxn];
int w[maxn][maxn];

struct node {
    int id, d;
    bool operator < (const node& p) const { return d < p.d; }
    bool operator > (const node& p) const { return d > p.d; }
};
int dis[maxn];
bool st[maxn];
int f[maxn];

```

```

void dijkstra(int _st) {
    memset(dis, 0x3f, sizeof(dis)), memset(st, false, sizeof(st));
    priority_queue<node, vector<node>, greater<node>> q;
    q.push({_st, 0}); dis[_st] = 0; f[_st] = 1;
    while (!q.empty()) {
        node u = q.top(); q.pop();
        int id = u.id, d = u.d;
        if (st[id]) continue;
        st[id] = true;

        for (eInfo it : vec[id]) {
            if (dis[it.to] > d+it.value) {
                dis[it.to] = d+it.value; q.push({it.to, dis[it.to]}); f[it.to] = f[id];
            } else if (dis[it.to] == d+it.value) f[it.to] += f[id];
        }
    }
}

int main()
{
    int n, m; cin >> n >> m;
    memset(w, 0x3f, sizeof(w));
    while (m -- ) {
        int a, b, c; cin >> a >> b >> c; w[a][b] = min(w[a][b], c);
    }

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (w[i][j] != inf) vec[i].push_back({j, w[i][j]});
        }
    }

    dijkstra(1);
    if (dis[n] == inf) cout << "No answer" << endl;
    else cout << dis[n] << " " << f[n] << endl;
    return 0;
}

```

Hopscotch Addict

分层图, 把 1 个点拆成 3 个点, 然后从起点往终点跑 bfs 求最短路

```

#include <bits/stdc++.h>

using namespace std;

int get_id(int a, int b) { return a*3+b; }

const int maxn = 3e5 + 100;
vector<int> vec[maxn];

```

```
int dis[maxn];

int bfs(int a, int b) {
    int _st = get_id(a,0), _ed = get_id(b,0);
    memset(dis, -1, sizeof(dis));

    queue<int> q; q.push(_st); dis[_st] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int i : vec[u]) {
            if (dis[i] == -1) q.push(i), dis[i] = dis[u]+1;
        }
    }

    return (dis[_ed]==-1 ? dis[_ed] : dis[_ed]/3);
}

int main()
{
    int n, m; cin >> n >> m;
    while (m -- ) {
        int a, b; cin >> a >> b;
        for (int i = 0; i < 3; ++i) {
            int a_id = get_id(a,i), b_id = get_id(b,(i+1)%3);
            vec[a_id].push_back(b_id);
        }
    }

    int a, b; cin >> a >> b;
    cout << bfs(a, b) << endl;
    return 0;
}
```